

Technical Feasibility & Step-by-Step Build Guide

AI-Powered Object Scanning & Guided Assistance App

1. Executive Summary

ScanAssist is an AI-powered mobile application that allows users to point their phone camera at any physical object, device, or appliance and receive real-time guided assistance through voice instructions and visual gesture overlays drawn directly on their photo. The app is designed to remove the friction of figuring out how to use, fix, or build anything, with a primary focus on elderly users in assisted living facilities.

This document outlines the complete technical architecture, build steps, tools, APIs, and timeline required to develop a functional hackathon prototype.

2. Core Features Overview

Feature	Description	Complexity
Object Recognition	Identify any scanned device or appliance via AI vision	Low
Step-by-Step Guidance	Verbal and text walkthrough for use, repair, or assembly	Low
Gesture Overlays	Arrows and circles drawn on the photo pointing to exact components	Medium-High
Voice Input & Output	Speak your question, hear the answer back	Low
Conversational Loop	Back-and-forth dialogue to handle follow-up questions	Medium
Safety Flags	Auto-detect dangerous tasks and recommend professionals	Low

3. Technology Stack

3.1 Frontend

- React Native (recommended for mobile) or React.js for a web-based prototype
- Expo for rapid mobile development without native build tools
- Fabric.js or HTML5 Canvas API for gesture overlay rendering on images
- Tailwind CSS for clean, accessible UI styling

3.2 AI & Intelligence Layer

- Claude API (claude-sonnet-4-6) for object recognition, reasoning, and step generation
- Claude Vision for analyzing uploaded or captured photos

- Prompt chaining for multi-turn conversational assistance

3.3 Voice Layer

- Web Speech API (built into all modern browsers) for speech-to-text input — completely free
- Web Speech Synthesis API for text-to-speech output — completely free
- ElevenLabs API (optional upgrade) for more natural, human-like voice output

3.4 Backend

- Node.js + Express for a lightweight API server
- Firebase or Supabase for user session and scan history storage
- Cloudinary or Firebase Storage for temporary image handling

3.5 Deployment

- Vercel or Netlify for fast frontend deployment
- Railway or Render for backend hosting
- All free tiers sufficient for a hackathon prototype

4. System Architecture

The application follows a five-stage pipeline for every user interaction:

Stage	What Happens	Technology Used
1. Capture	User opens app and takes photo or uploads image of object	Device Camera / React Native
2. Recognize	Image sent to Claude Vision API which identifies the object and its components	Claude API + Vision
3. Intent	User states what they need (fix / use / build) via voice or text	Web Speech API
4. Guide	Claude generates step-by-step instructions with component location data	Claude API
5. Overlay	App parses coordinate hints from AI response and draws arrows on the image	Canvas API / Fabric.js

5. Step-by-Step Build Guide

The following section walks through the complete build process from environment setup to a deployable prototype. Estimated total build time for a two-person team is 10 to 14 hours.

PHASE 1: Environment Setup (1 hour)

Step 1 — Initialize the Project

1. Install Node.js (v18+) and npm if not already installed
2. Run: `npx create-react-app scanassist` or `npx create-expo-app scanassist` for mobile
3. Install dependencies: `npm install axios fabric react-webcam`
4. Create a `.env` file and add your Claude API key:
`REACT_APP_CLAUDE_KEY=your_key_here`

Step 2 — Set Up Project Structure

- `src/components/` for UI components (CameraView, OverlayCanvas, StepGuide, VoiceButton)
- `src/services/` for API calls (claudeService.js, speechService.js)
- `src/utils/` for helper functions (imageUtils.js, overlayParser.js)

PHASE 2: Camera & Image Capture (1.5 hours)

Step 3 — Build the Camera Component

5. Use the react-webcam library to access the device camera in browser
6. Add a large, single capture button centered on screen (accessibility first)
7. On capture, convert the image to base64 format for API transmission
8. Display a preview of the captured image before sending for confirmation
9. Add a retake button so users are never stuck with a bad photo

Step 4 — Add Image Quality Guidance

10. Before capture, show a simple overlay: green border = good framing, red = too close or too dark
11. Use basic brightness detection on the camera feed to warn if lighting is poor
12. Add a gentle prompt: 'Move closer to the object for best results'

PHASE 3: AI Object Recognition (2 hours)

Step 5 — Connect to Claude Vision API

13. In `claudeService.js`, write a function `sendImageToClaudeVision(base64Image)`
14. Pass the base64 image to the Claude API messages endpoint with `media_type image/jpeg`

15. Include a system prompt instructing Claude to identify the object and list its key components with approximate locations (e.g. top-left, center, bottom-right)
16. Parse the response to extract: object name, object category, component list with locations

Step 6 — Display Recognition Result

17. Show the identified object name prominently: 'This looks like a Bosch washing machine'
18. Add a confirmation prompt: 'Is this correct?' with Yes / No buttons
19. If No, allow the user to type or speak a correction
20. On confirmation, proceed to intent selection

PHASE 4: Intent & Instruction Engine (2 hours)

Step 7 — Intent Selection Screen

21. Present four large, icon-based buttons: How do I use this / How do I fix this / How do I build this / Tell me more about this
22. Support voice input as alternative to button tap
23. Pass selected intent + object context to Claude for instruction generation

Step 8 — Generate Step-by-Step Instructions

24. Send a prompt to Claude: 'You are a helpful assistant. The user has scanned a [object name]. They want to know how to [intent]. Give clear step-by-step instructions. For each step that references a physical component, include its location in the image in square brackets e.g. [bottom-left panel]'
25. Stream the response word-by-word using Claude's streaming API so text appears instantly
26. Display each step as a card the user can tap to mark complete
27. Parse location tags from each step for the overlay engine

Step 9 — Build the Conversational Loop

28. After each step, show a microphone button: 'Ask a question about this step'
29. Append user follow-up to the conversation history and send back to Claude
30. Maintain full message history so Claude retains context across the entire session
31. Add quick-reply buttons: 'I don't have that tool' / 'I don't see that part' / 'Slow down'

PHASE 5: Gesture Overlay Engine (3 hours)

Step 10 — Set Up Canvas Layer

32. Use Fabric.js to create a canvas element positioned directly over the captured photo
33. Set canvas dimensions to match image dimensions exactly
34. Ensure canvas is transparent so the photo shows through

Step 11 — Parse Location Data from Claude

35. Write a parser in `overlayParser.js` that reads location tags from Claude's response e.g. [bottom-left panel], [top-right corner], [center dial]
36. Map text locations to approximate pixel regions: top = 0-33% of height, left = 0-33% of width etc.
37. Generate an (x, y) coordinate and radius for each referenced component

Step 12 — Draw Overlays

38. For each step, draw a pulsing circle around the relevant component using Fabric.js
39. Add a numbered label matching the step number (1, 2, 3...)
40. Draw an arrow from the label to the component center
41. Animate the circle with a gentle pulse to draw the eye
42. Clear and redraw overlays when the user moves to the next step

PHASE 6: Voice Layer (1 hour)

Step 13 — Text to Speech Output

43. Use the browser's built-in `window.speechSynthesis` API
44. Auto-read each step aloud when it appears on screen
45. Add a speaker icon to replay any step on demand
46. Set speech rate to 0.85 (slightly slower than default) for elderly users

Step 14 — Speech to Text Input

47. Use `window.SpeechRecognition` API for voice input
48. Activate on microphone button press, deactivate on release or silence
49. Transcribe speech to text and display in the input field before sending
50. Add visual feedback (waveform animation) so users know the mic is active

PHASE 7: Safety & Polish (1 hour)

Step 15 — Safety Detection

51. Add a keyword check in the Claude system prompt: if the task involves electricity, gas, structural elements, or medical devices, append a safety warning
52. Display a prominent yellow warning card: 'This task may require a professional. Proceed carefully.'
53. For high-risk tasks, offer a 'Find a local technician' button (links to Google search)

Step 16 — Accessibility Optimizations

- 54. Minimum font size 18px throughout the app
- 55. All buttons minimum 60x60px tap target
- 56. High contrast mode toggle in settings
- 57. Haptic feedback on button taps (mobile)
- 58. All instructions available in both text and audio simultaneously

6. API Reference & Cost Estimates

API / Service	Purpose	Free Tier	Hackathon Cost
Claude API (Anthropic)	Object recognition + instruction generation	\$5 free credit on signup	~\$2-5 for full hackathon
Web Speech API	Voice input and output	100% free, built into browser	\$0
Firebase	Image storage + session data	5GB storage free	\$0
Vercel	Frontend deployment	Free tier	\$0
Railway	Backend hosting	Free tier (500 hrs/month)	\$0
ElevenLabs (optional)	Premium voice output	10,000 chars/month free	\$0

Total estimated cost for a hackathon prototype: \$2 to \$5 (Claude API usage only). All other services are free.

7. Build Timeline

Phase	Task	Time	Owner Suggestion
1	Environment setup & project scaffold	1 hour	Both team members
2	Camera capture & image preview	1.5 hours	Frontend developer
3	Claude Vision API integration	2 hours	Backend developer
4	Intent selection + instruction engine	2 hours	Backend developer
5	Gesture overlay engine	3 hours	Frontend developer
6	Voice input and output	1 hour	Frontend developer
7	Safety flags + accessibility polish	1 hour	Both team members

Phase	Task	Time	Owner Suggestion
8	End-to-end testing + demo prep	1.5 hours	Both team members
	TOTAL	13 hours	2-person team

8. Known Technical Risks & Mitigations

Risk	Likelihood	Mitigation
Gesture overlay coordinates inaccurate	High	Use regional zones (top/center/bottom-left/right) rather than pixel-perfect targeting
Object misidentified by vision model	Medium	Always confirm with user before proceeding. Show 'Is this correct?' screen
Poor image quality breaks recognition	Medium	Add real-time lighting guidance before capture. Prompt user to move closer
API latency feels slow	Medium	Stream responses word-by-word + audio filler ('Got it, looking that up...')
No internet connection	Low	Cache 20 most common appliance guides locally for offline fallback
Voice recognition fails in noisy environment	Medium	Always provide text input as parallel fallback option

9. Demo Strategy

The most effective hackathon demo follows this sequence:

59. Bring a real physical object on stage — ideally something with multiple visible components like a coffee maker, power strip, or small appliance
60. Open the app live on a phone or laptop
61. Scan the object in real time — let the audience watch it identify
62. Say: 'How do I fix this?' via voice
63. Watch the step-by-step guide appear with gesture overlays pointing to real components
64. Ask a follow-up question mid-flow to demonstrate the conversational loop

This 90-second demo sequence is self-explanatory, visually impressive, and requires no slides to land the concept.

10. Future Roadmap (Post-Hackathon)

Version 2.0 — 3 Months

- AR overlay using device gyroscope to keep arrows anchored in real-time video, not just a still photo
- Multi-language support — auto-detect user language and respond accordingly
- Offline mode with cached guides for 50 most common household appliances

Version 3.0 — 6 Months

- Facility dashboard for assisted living operators to monitor resident usage and common issues
- Manufacturer white-label API so brands can embed ScanAssist into their own support apps
- Community-contributed fix guides — verified users can submit and rate repair instructions

Version 4.0 — 12 Months

- Native OS integration partnership with Apple Accessibility or Google Lens ecosystem
- Insurance and Medicare Advantage pilot programs
- Technician referral marketplace for tasks beyond DIY scope

Built for impact. Designed for everyone.